AIMS 5710: Deep Learning Fundamentals and Theories

Neural Networks

WANG Liming

Office: SHB 601F
e-mail: lmwang@se.cuhk.edu.hk
web: https://www.se.cuhk.edu.hk

Department of SEEM
The Chinese University of Hong Kong
Slides adapted from course materials originally developed by Prof. LI Hongsheng.

Outline

1. Computational graph of linear models
2. Fully-connected layers
3. Some other layer types

1 Computational graph of linear models

2 Fully-connected layers

3 Some other layer types

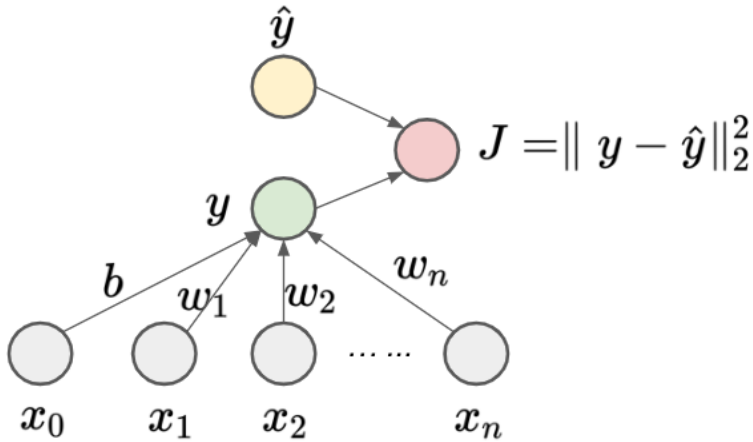
Graphical representations of linear regression

- Recall that linear regression and its cost function can be formulated as

$$y = w_1 x_1 + w_2 x_2 + \cdots w_n x_n + b$$

$$J(w_1, \cdots, w_n, b) = \|y - \hat{y}\|_2^2$$

- We here represent linear regression as a computational graph
- Each input node represents one individual feature value $x_1, x_2, \cdots, x_n$ of one individual feature vector $x = \{x_1, x_2, \cdots, x_n\}$
- A constant input node $x_0 = 1$ is also utilized. Weights associated with input nodes are denoted as $w_1, w_2, \cdots, w_n$ and b
- The ground-truth label is denoted as $\hat{y}$



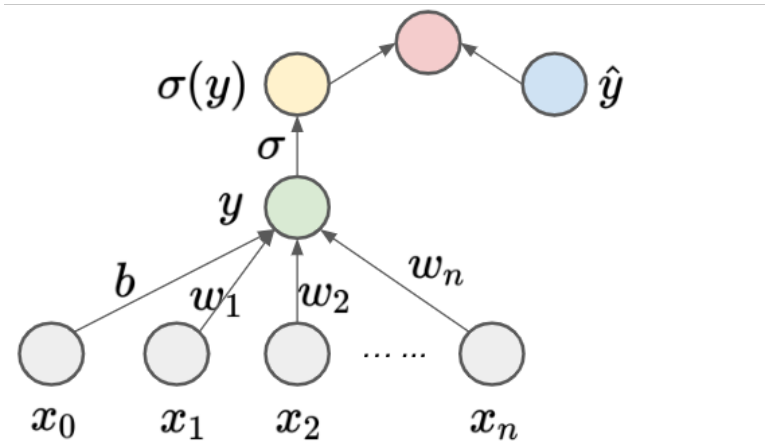
Graphical representations of logistic regression

- Similarly, the logistic classification and its cost function are

$$\sigma(y) = \sigma(w_1 x_1 + w_2 x_2 + \cdots w_n x_n + b)$$
$$J(w_1, \cdots, w_n, b) = -\hat{y} \log \sigma(y) - (1 - \hat{y}) \log(1 - \sigma(y))$$

- We here represent linear regression as a computational graph
- Each input node represents one individual feature value $x_1, x_2, \cdots, x_n$ of one individual sample $x = \{x_1, x_2, \cdots, x_n\}$
- A constant input node $x_0 = 1$ is also utilized. Weights associated with input nodes are denoted as $w_1, w_2, \cdots, w_n$ and b
- The ground-truth label (either 0 or 1) is denoted as $\hat{y}$

$$J = -\hat{y} \log \sigma(y) - (1 - \hat{y}) \log(1 - \sigma(y))$$



Graphical representations of C-class logistic regression

- Similarly, the C-class logistic classification and its cost function are

$$\begin{aligned} y_1 &= w_{11}x_1 + w_{12}x_2 + \cdots w_{1n}x_n + b_1 \\ y_2 &= w_{21}x_1 + w_{22}x_2 + \cdots w_{2n}x_n + b_2 \\ &\cdots \\ y_C &= w_{C1}x_1 + w_{C2}x_2 + \cdots w_{Cn}x_n + b_C \end{aligned}$$

- $y_1, y_2, \cdots, y_C$ are then normalized by the following softmax function

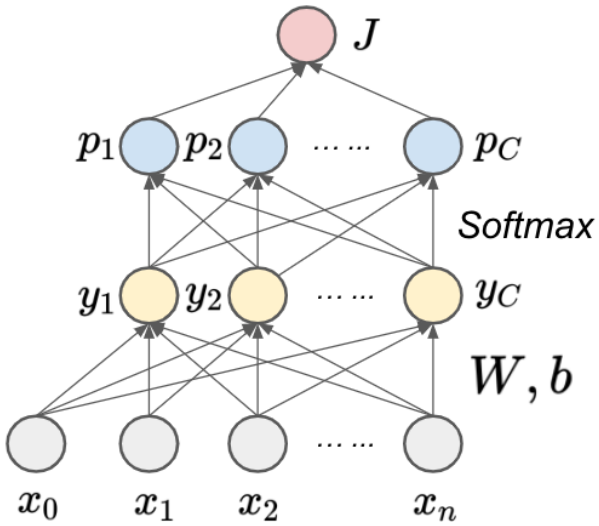
$$p_k = \frac{\exp(y_k)}{\sum_{i=1}^{C} \exp(y_i)}$$

- The loss functions are denoted as

$$J(W, b) = -\sum_{i=1}^{C} \hat{y}_i \log p_i$$

Fully-connected layer in neural networks

- The computational graph of multi-class (C-class) logistic classification algorithm can be drawn as



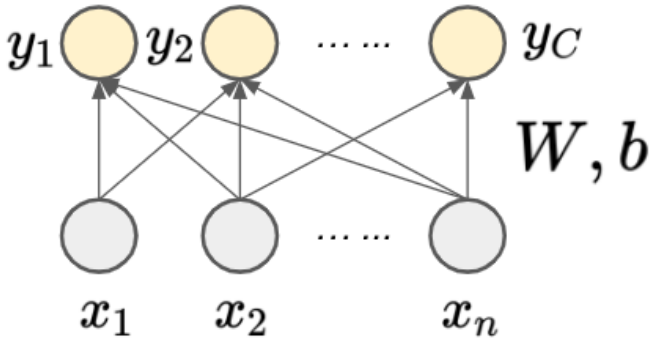
1 Computational graph of linear models

2 Fully-connected layers

3 Some other layer types

Fully-connected (linear) layer in neural networks

- The linear calculation to calculate $y_1, y_2, \cdots, y_C$ from $x_1, x_2, \cdots, x_n$ are named as *fully-connected layer* in neural networks
- It is one of the basic structure blocks in neural networks



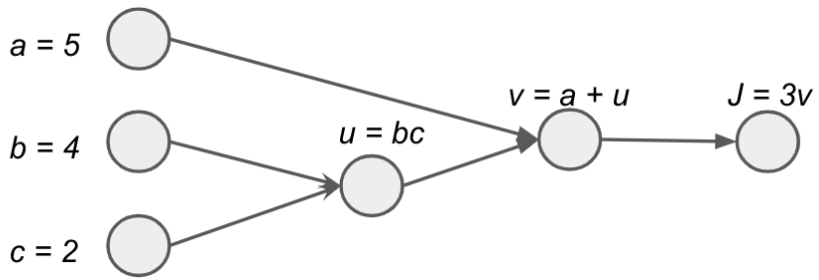
- The linear computation between x and y can be denoted as a matrix-vector multiplication $y = Wx + b$, where $W \in \mathbb{R}^{C \times n}$ and $b \in \mathbb{R}^C$ are learnable parameters and $x \in \mathbb{R}^n$ is the feature vector of one sample

$$W = \begin{bmatrix} w_{11} & w_{12} & \cdots & w_{1n} \\ w_{21} & w_{22} & \cdots & w_{2n} \\ \cdots & & & \\ w_{C1} & w_{C2} & \cdots & w_{Cn} \end{bmatrix} \quad b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_C \end{bmatrix} \quad x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

Computational graph

- Computational graph is a graphical representation of a function composition
- Example

$$u = bc, \quad v = a + u, \quad J = 3v$$

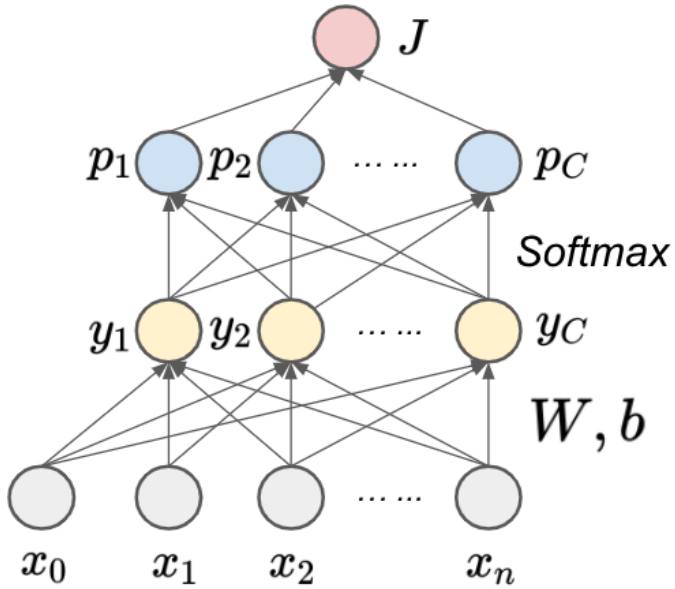


- The derivatives can be calculated backward sequentially without redudant computation

$$\frac{\partial J}{\partial v}, \quad \frac{\partial J}{\partial u} = \frac{\partial J}{\partial v}\frac{\partial v}{\partial u}, \quad \frac{\partial J}{\partial a} = \frac{\partial J}{\partial v}\frac{\partial v}{\partial a}, \quad \frac{\partial J}{\partial b} = \frac{\partial J}{\partial u}\frac{\partial u}{\partial b}, \quad \frac{\partial J}{\partial c} = \frac{\partial J}{\partial u}\frac{\partial u}{\partial c}$$

Gradients of fully connected layer

- The softmax or sigmoid functions are usually called the **non-linearity (or activation)** function in neural networks
- Recall that we have the following computational graph



- Given the loss function w.r.t. W, b

$$J(W, b) = -\sum_{i=1}^{C} \hat{y}_i \log p_i$$

- Our ultimate goal is to obtain $\frac{\partial J}{\partial W_{ij}}$ and $\frac{\partial J}{\partial b_i}$ to train the neural network

Gradients of cross-entropy loss

- Recall that the derivatives along computational graph can be calculated sequentially
- Eventually, we try to obtain

$$\frac{\partial J}{\partial p_i} \quad \frac{\partial J}{\partial y_i} \quad \frac{\partial J}{\partial W_{ij}} \quad \frac{\partial J}{\partial b_i}$$

- We therefore can calculate the following gradients sequentially and use chain rule to obtain the above gradients

$$\frac{\partial J}{\partial p_i} \quad \frac{\partial p_i}{\partial y_i} \quad \frac{\partial y_i}{\partial W_{ij}} \quad \frac{\partial y_i}{\partial b_i}$$

- Gradients of cross-entropy loss layer

$$\frac{\partial J}{\partial p_i} = \begin{cases} -\frac{1}{p_i} & \hat{y}_i = 1 \\ 0 & \hat{y}_i = 0 \end{cases}$$

Gradients of softmax layer

- We are interested in calculating the gradients

$$\frac{\partial p_i}{\partial y_j} = \frac{\partial \frac{e^{y_i}}{\sum_{k=1}^{C} e^{y_k}}}{\partial y_j}$$

- We will be using quotient rule of derivatives. For $f(x) = \frac{g(x)}{h(x)}$,

$$f'(x) = \frac{g'(x)h(x) - h'(x)g(x)}{[h(x)]^2}$$

where in our case, we have

$$g = e^{y_i}, \qquad h = \sum_{k=1}^{C} e^{y_k}$$

Gradients of softmax layer

- If $i = j$

$$\begin{aligned}\frac{\partial p_i}{\partial y_j} = \frac{\partial \frac{e^{y_i}}{\sum_{k=1}^{C} e^{y_k}}}{\partial y_j} &= \frac{e^{y_i} \sum_{k=1}^{C} e^{y_k} - e^{y_j} e^{y_i}}{\left(\sum_{k=1}^{C} e^{y_k}\right)^2} \\ &= \frac{e^{y_i} \left(\sum_{k=1}^{C} e^{y_k} - e^{y_j}\right)}{\left(\sum_{k=1}^{C} e^{y_k}\right)^2} \\ &= \frac{e^{y_i}}{\sum_{k=1}^{C} e^{y_k}} \times \frac{\left(\sum_{k=1}^{C} e^{y_k} - e^{y_j}\right)}{\sum_{k=1}^{C} e^{y_k}} \\ &= p_i \left(1 - p_j\right)\end{aligned}$$

- If $i \neq j$

$$\begin{aligned}\frac{\partial p_i}{\partial y_j} = \frac{\partial \frac{e^{y_i}}{\sum_{k=1}^{C} e^{y_k}}}{\partial y_j} &= \frac{0 - e^{y_j} e^{y_i}}{\left(\sum_{k=1}^{C} e^{y_k}\right)^2} \\ &= \frac{-e^{y_j}}{\sum_{k=1}^{C} e^{y_k}} \times \frac{e^{y_i}}{\sum_{k=1}^{C} e^{y_k}} \\ &= -p_j \cdot p_i\end{aligned}$$

Gradients of softmax layer

- Therefore, the gradients of the softmax layer can be defined as

$$\frac{\partial p_i}{\partial y_j} = \begin{cases} p_i \left(1 - p_j\right) & \text{if} \quad i = j \\ -p_j \cdot p_i & \text{if} \quad i \neq j \end{cases}$$

- Combining gradients of the two layers, cross-entropy layer and softmax layer, the gradient of J w.r.t. y_i can be calculated as $\frac{\partial J}{\partial y_j} = \frac{\partial J}{\partial p_i} \frac{\partial p_i}{\partial y_j}$ for $\hat{y}_i = 1$
- If $i = j$ and $\hat{y}_i = 1$,

$$\frac{\partial J}{\partial y_j} = -(1 - p_i),$$

If $i \neq j$ and $\hat{y}_i = 1$,

$$\frac{\partial J}{\partial y_j} = p_j$$

Gradients of fully-connected layer

- Recall that a fully-connected layer is calculated as

$$y = Wx + b$$

$$y_1 = w_{11}x_1 + w_{12}x_2 + \cdots w_{1n}x_n + b_1$$

$$y_2 = w_{21}x_1 + w_{22}x_2 + \cdots w_{2n}x_n + b_2$$

$$\cdots$$

$$y_C = w_{C1}x_1 + w_{C2}x_2 + \cdots w_{Cn}x_n + b_C$$

- Gradients w.r.t. w_{ij} and b_i can be calculated as

$$\frac{\partial y_i}{\partial w_{ij}} = x_j \qquad \frac{\partial y_i}{\partial b_i} = 1$$

- Multiplying the gradients from the above layer according to chain rule of derivatives results in

$$\frac{\partial J}{\partial w_{ij}} = \frac{\partial J}{\partial y_i}\frac{\partial y_i}{\partial w_{ij}} = \frac{\partial J}{\partial y_i}x_j \qquad \frac{\partial J}{\partial b_i} = \frac{\partial J}{\partial y_i}\frac{\partial y_i}{\partial b_i} = \frac{\partial J}{\partial y_i}$$

- In matrix and vector format, we have

$$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial y}x^T \text{(outer product of the two vectors)}, \qquad \frac{\partial J}{\partial b} = \frac{\partial J}{\partial y}$$

Gradients of the fully-connected layer

- We can further calculate gradients of fully-connected layers w.r.t. inputs $x_1, x_2, \cdots, x_n$
- Gradients of the fully-connected layer can be calculated as

$$\frac{\partial y_i}{\partial x_j} = w_{ij}$$

- Gradients of J w.r.t. x_i therefore can be calculated as

$$\frac{\partial J}{\partial x_j} = \sum_{i=1}^{C} \frac{\partial J}{\partial y_i} \frac{\partial y_i}{\partial x_j} = \sum_{i=1}^{C} \frac{\partial J}{\partial y_i} w_{ij}$$

Converting this into a vector format, we have

$$\frac{\partial J}{\partial x} = W^T \frac{\partial J}{\partial y}$$

Forward computation and back-propagation

- Each layer's calculation can be categorized into forward and backward calculation
- Forward computation: for calculating classification probabilities from bottom layer to top layers sequentially
- Backward computation (back-propagation): for calculating gradients for parameter update from top layer to bottom layers sequentially

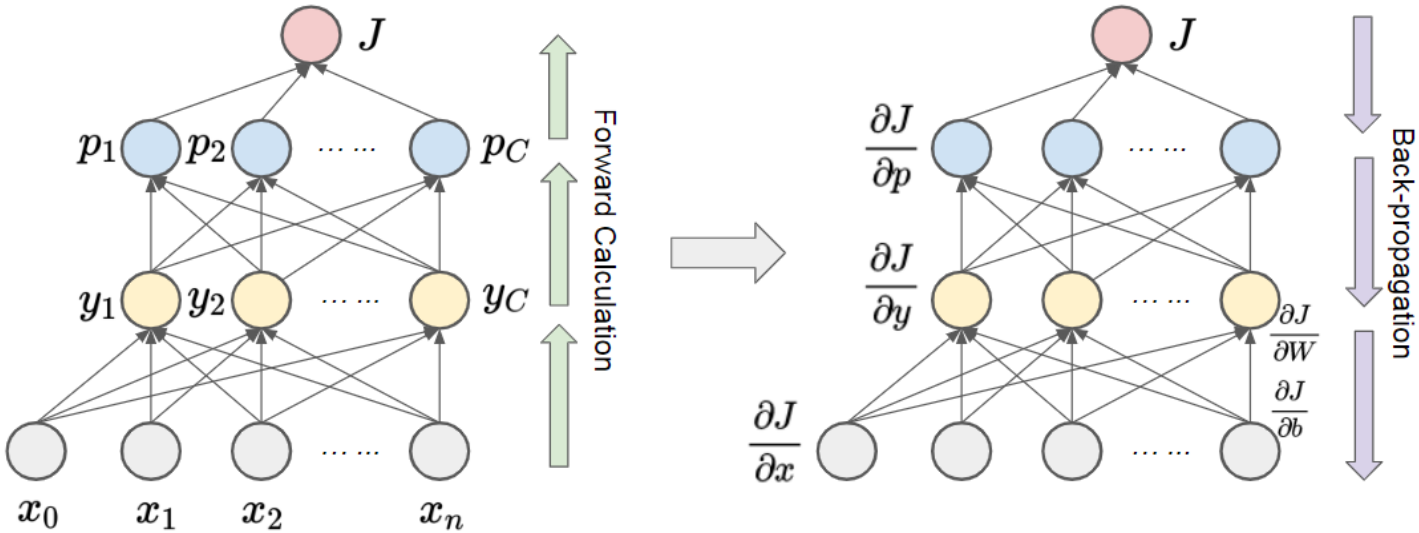
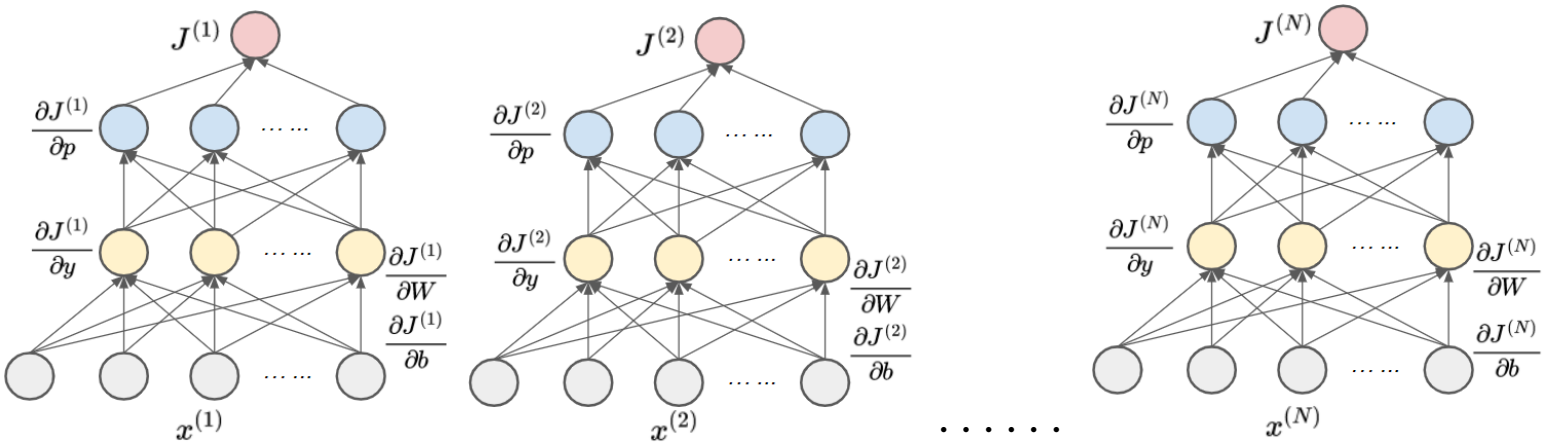


Figure: In each training iteration, (1) forward computation from bottom to top and then (2) back-propagation from top to bottom.

Gradients of a mini-batch of samples

- Recall that we mentioned that for large-scale data, the neural networks are generally trained with Stochastic Gradient Descent
- Stochastic Gradient Descent calculates derivatives J w.r.t. W, b using a mini-batch of training samples $\{x^{(1)}, x^{(2)}, \cdots, x^{(N)}\}$



- The gradients for updating parameters will be calculated as the average of the gradients of the mini-batch with batch size N,

$$J = \frac{1}{N}\sum_{i=1}^{N}\left(J^{(1)} + J^{(2)} + \cdots + J^{(N)}\right)$$

$$\frac{\partial J}{\partial W} = \frac{1}{N}\sum_{i=1}^{N}\frac{\partial J^{(i)}}{\partial W} \qquad \frac{\partial J}{\partial b} = \frac{1}{N}\sum_{i=1}^{N}\frac{\partial J^{(i)}}{\partial b}$$

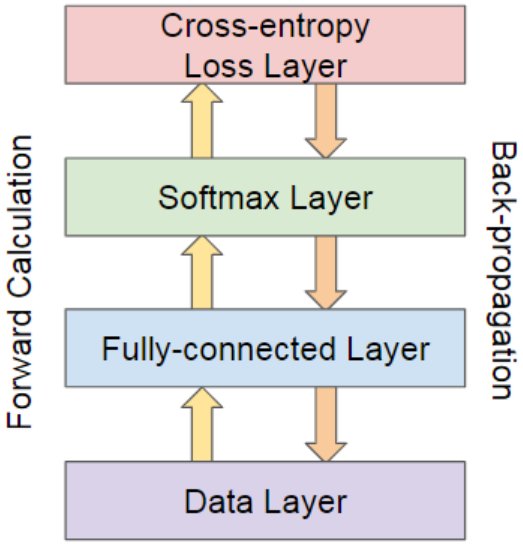
Summary of fully-connected, softmax, and cross-entropy loss layers

- Fully-connected layer
  - Input: $x = [x_1, x_2, \cdots, x_n]$, output: $y = [y_1, y_2, \cdots, y_C]$
  - Learnable parameters: W and b
  - Forward input: x, foward output: $y = Wx + b$
  - Backward input: $\dfrac{\partial J}{\partial y}$
  - Backward output: $\dfrac{\partial J}{\partial x} = W^T \dfrac{\partial J}{\partial y}$, $\dfrac{\partial J}{\partial W} = \dfrac{\partial J}{\partial y} x^T$, $\dfrac{\partial J}{\partial b} = \dfrac{\partial J}{\partial y}$
- Cross-entropy loss layer
  - Input: $p = [p_1, p_2, \cdots, p_C]$, $\hat{y} = [\hat{y}_1, \hat{y}_2, \cdots, \hat{y}_C]$, output J
  - Learnable parameters: None
  - Forward input: p, $\hat{y}$, foward output: $-\sum_{i=1}^{C} \hat{y}_i \log p_i$
  - Backward output: $\dfrac{\partial J}{\partial p_i} = \begin{cases} -\dfrac{1}{p_i} & \hat{y}_i = 1 \\ 0 & \hat{y}_i = 0 \end{cases}$
- A neural network can be considered as a structure consisting of the basic layers

Summary of fully-connected, softmax, and cross-entropy loss layers

- Softmax layer
  - Input: $y = [y_1, y_2, \cdots, y_C]$, output: $p = [p_1, p_2, \cdots, p_C]$
  - Learnable parameters: None
  - Foward input: y, forward output: $p_i = \dfrac{\exp(y_i)}{\sum_{j=1}^{C} \exp(y_j)}$
  - Backward input: $\left[\frac{\partial J}{\partial p_1}, \frac{\partial J}{\partial p_2}, \ldots, \frac{\partial J}{\partial p_C}\right]$, Backward output:

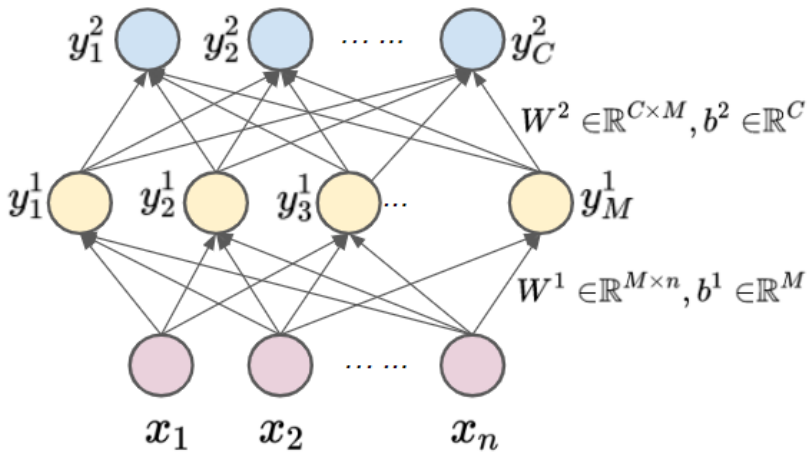
$$\frac{\partial J}{\partial y_i} = \frac{\partial J}{\partial p_i} p_i(1 - p_i) - \sum_{j \neq i} \frac{\partial J}{\partial p_j} p_j p_i$$



Multi-Layer Perceptron

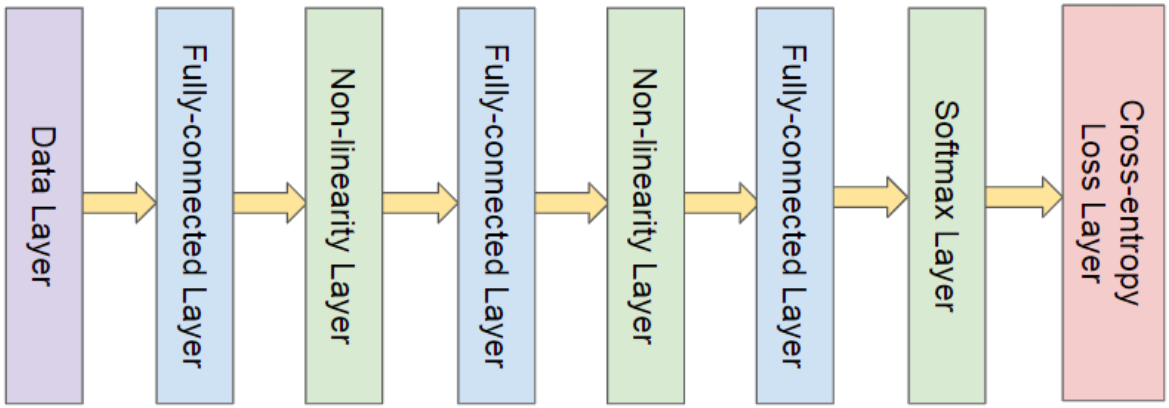
- A neural network generally consists of multiple stacked fully-connected (linear) stacked together, where each layer has their independent parameters to learn (in general cases)
- We generally do not draw non-linearity function layers between and after fully-connected layers and do not draw $x_0, y_0^1, y_0^2, y_0^3, \cdots$
- However, the multiple fully connected layer has to be separated by non-linearity layers (e.g., softmax or sigmoid layers). Otherwise, multiple stacked fully-connected layer is equivalent to ONE fully-connected layer

$$y^2 = W^2(W^1 x + b^1) + b^2 = \left[W^2 W^1\right] x + \left[W^2 b^1 + b^2\right]$$



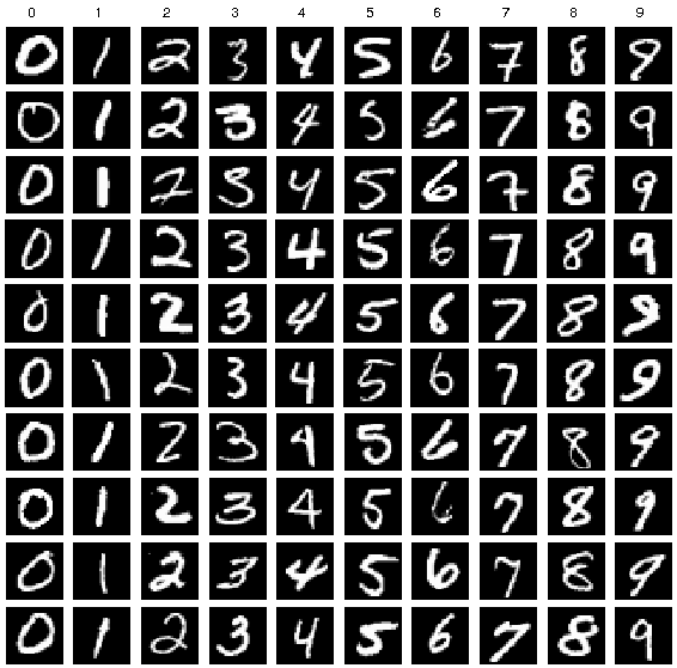
Multi-layer Perceptron

- Generally, a single linear layer with non-linearity function (e.g., logistic classification) does not have enough capacity to model the underlying function
- Neural networks with > 2 fully-connected layers can approximate any highly non-linear function
- A 3-layer Multi-Layer Perceptron (MLP) can be illustrated below



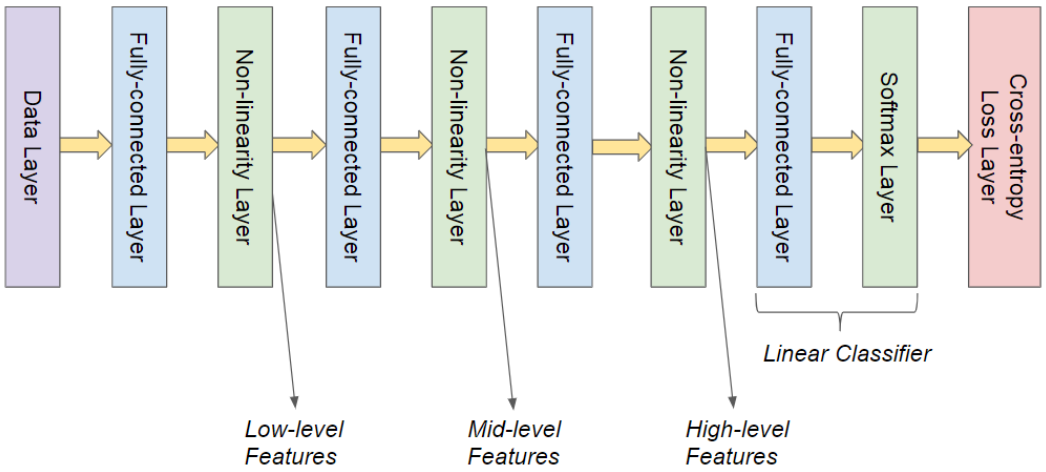
The MNIST dataset

- The MNIST dataset is a large database of handwritten digits that is commonly used for evaluating different machine learning algorithms
- It contains 60,000 training images and 10,000 testing images
- Each image is of size 32×32
- To use MLP to classify the digits, the 32×32 images can be vectorized into $32 \times 32 = 1024$ feature vectors as inputs



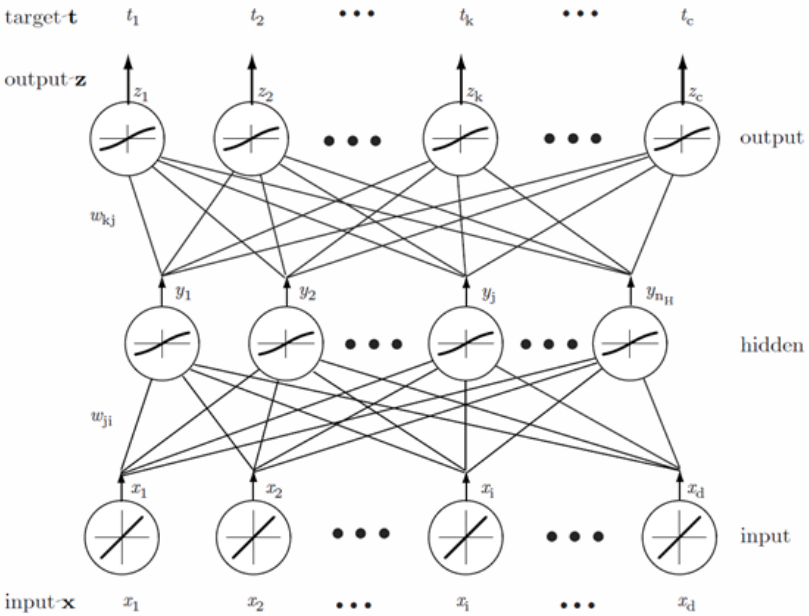
Deeply learned feature representations

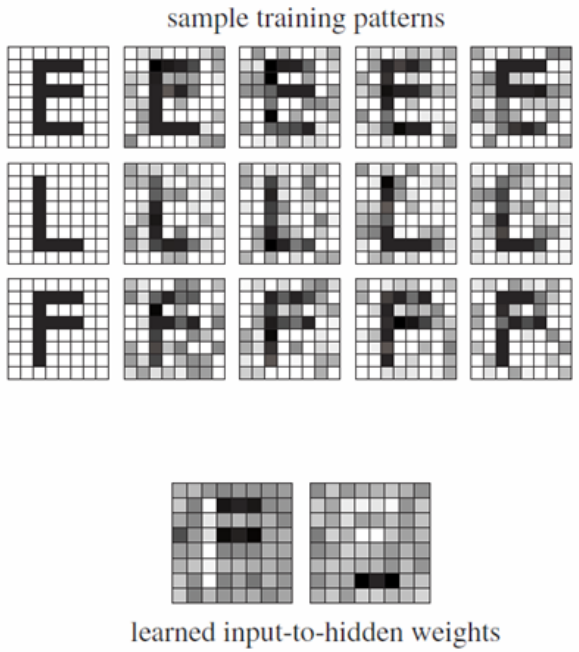
- Recall that in the begin of the course, we claimed that deep neural networks are "learning" features instead of using manually designed features
- The last fully-connected layer with the non-linearity function layer can be considered as a linear classifier
- All the previous neural layers can be considered as a series of transformations that gradually transform the input features into linearly separable features
- The low-level features captures more general information of samples of all classes
- The high-level features are closer to the final task



The learned weights

- The learned weights of each low-level neuron capture certain general patterns of all samples





(Duda et al. Pattern Classification 2000)

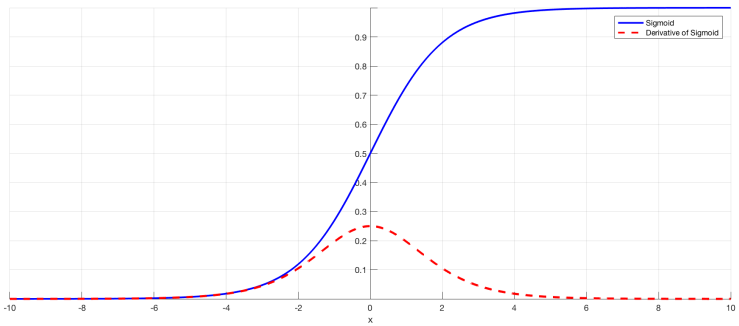
1 Computational graph of linear models

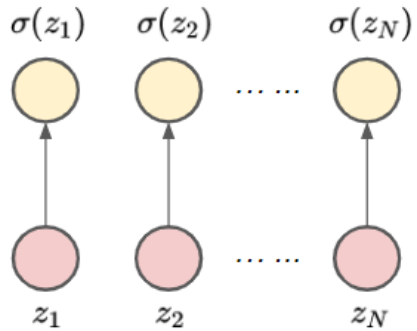
2 Fully-connected layers

3 Some other layer types

Non-linearity layers

- Sigmoid (function) layer
  - Unlike softmax function, the sigmoid function only takes one value as input and output one value each time
  - Input: $z = [z_1, z_2, \cdots, z_N]$, forward output: $y_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}$ for $i = 1, 2, \cdots, N$
  - Backward input: $\frac{\partial J}{\partial y_i}$ for $i = 1, 2, \cdots, N$, backward output: $\frac{\partial J}{\partial y_i} \cdot \sigma(z_i)(1 - \sigma(z_i))$ for $i = 1, 2, \cdots, N$
  - Use scenarios:
    - Back in 1990s-2000s, it was one of the most popular non-linearity function between fully connected layers
    - Can be used as the last layer of binary classification
    - Can be used to gate the information flow through another neuron





Non-linearity layers

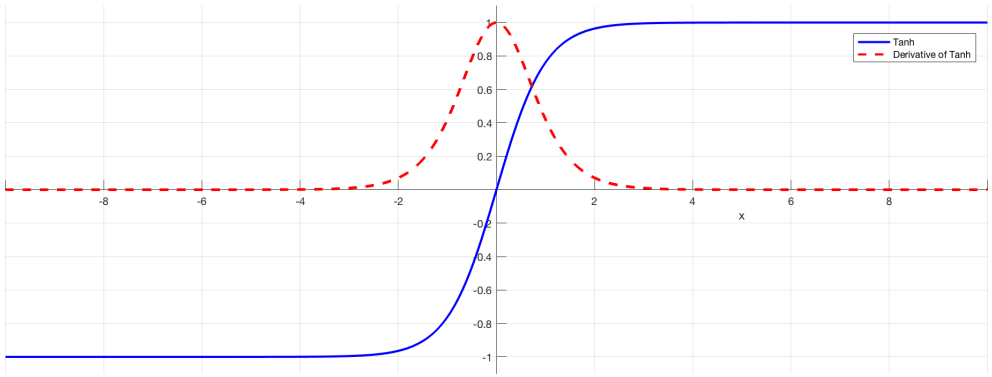
- Tanh (hyperbolic tangent function) layer
  - Sigmoid function maps $[-\infty, \infty]$ to $[0, 1]$, hyperbolic tangent function maps $[-\infty, \infty]$ to $[-1, 1]$
  - Forward input: $z = [z_1, z_2, \cdots, z_N]$, forward output:

$$g_{\tanh}(z) = \frac{\mathrm{e}^{z} - \mathrm{e}^{-z}}{\mathrm{e}^{z} + \mathrm{e}^{-z}}$$

  - Backward input: $\frac{\partial J}{\partial y_i}$ for $i = 1, 2, \cdots, N$, backward output:

$$\frac{\partial J}{\partial y_i} \cdot (1 - \tanh^2(z))$$

    for $i = 1, 2, \cdots, N$
  - It is now much less frequently used compared with sigmoid function



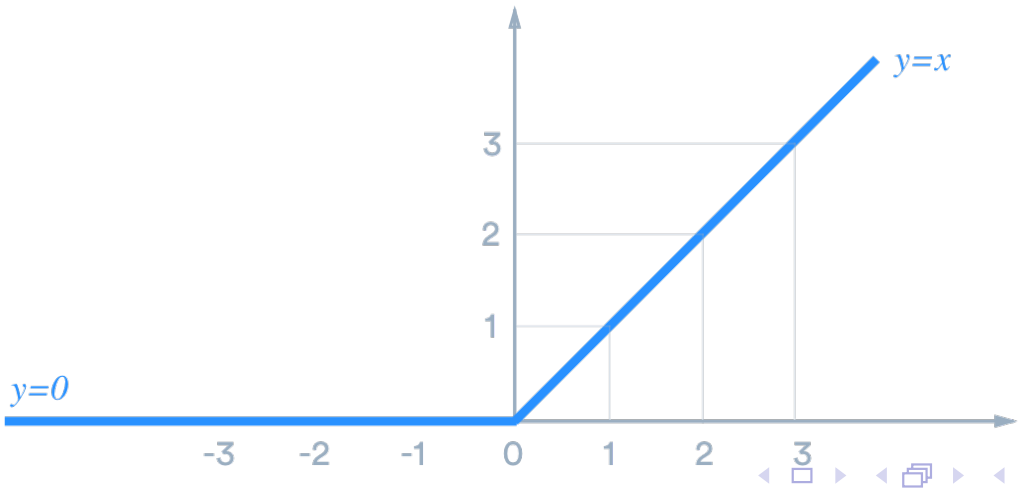
Non-linearity layers

- ReLU (Rectified Linear Unit) layer
  - One of the most frequently used non-linear function since 2012, because of its fast convergence rate
  - Forward input: $x = [x_1, x_2, \cdots, x_n]$; forward output:

$$y_i = \max(0, x_i) \text{ for } i = 1, 2, \cdots, n$$

  - Backward input: $\dfrac{\partial J}{\partial y_i}$ for $i = 1, 2, \cdots, N$; backward output:

$$\frac{\partial J}{\partial x_i} = \begin{cases} \frac{\partial J}{\partial y_i} & \text{if } x_i > 0 \\ 0 & \text{otherwise} \end{cases}$$



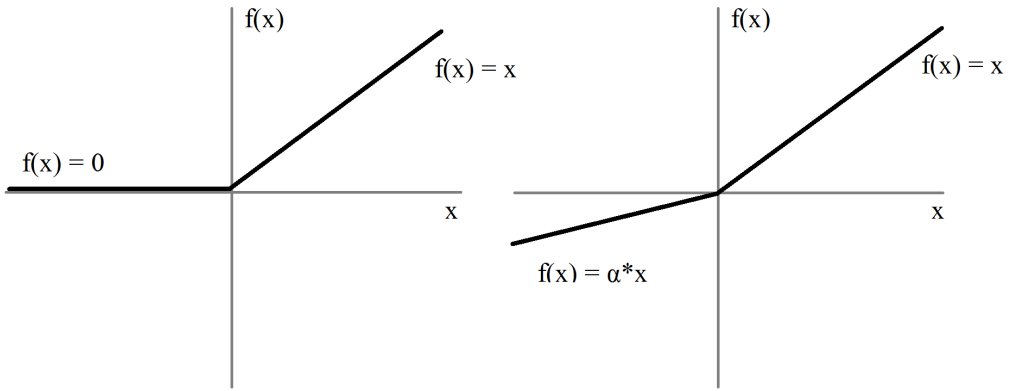
Non-linearity layers

- Leaky ReLU (Rectified Linear Unit) layer
  - Leaky ReLU is an improved version of the ReLU layer. It solves the problem of ReLU of having no gradients when the input is less than 0
  - Forward input: $x = [x_1, x_2, \cdots, x_n]$; forward output:

$$y_i = \begin{cases} \alpha x_i & \text{if } x_i < 0 \\ x_i & \text{if } x_i \geq 0 \end{cases} \quad \text{for } i = 1, 2, \cdots, n$$

    where α is a constant
  - Backward input: $\dfrac{\partial J}{\partial y_i}$ for $i = 1, 2, \cdots, n$; backward output:

$$\frac{\partial J}{\partial x_i} = \begin{cases} \alpha \dfrac{\partial J}{\partial y_i} & \text{if } x_i < 0 \\ \dfrac{\partial J}{\partial y_i} & \text{if } x_i \geq 0 \end{cases}$$



Non-linearity layers

- PReLU layer
  - PReLU takes one step further by making the coefficient of leakage α to be learned during network training
  - Forward input: $x = [x_1, x_2, \cdots, x_n]$; forward output:

$$y_i = \begin{cases} \alpha x_i & \text{if } x_i < 0 \\ x_i & \text{if } x_i \geq 0 \end{cases} \quad \text{for } i = 1, 2, \ldots, n$$

    where α is a learnable constant
  - Backward input: $\dfrac{\partial J}{\partial y_i}$ for $i = 1, 2, \ldots, n$; backward output:

$$\frac{\partial J}{\partial x_i} = \begin{cases} \alpha \frac{\partial J}{\partial y_i} & \text{if } x_i < 0 \\ \frac{\partial J}{\partial y_i} & \text{if } x_i \geq 0 \end{cases}$$

  - Parameter gradients:

$$\frac{\partial J}{\partial \alpha} = \sum_{i=1}^{n} \mathbf{1}(x_i < 0) x_i \cdot \frac{\partial J}{\partial y_i}$$

Loss layers

- Mean Squared Error (MSE)/L2 loss layer
  - Generally used for regression problem
  - Forward inputs: $z^{(1)}, z^{(2)}, \cdots, z^{(N)}$ and ground-truth $\hat{z}^{(1)}, \hat{z}^{(2)}, \cdots, \hat{z}^{(N)}$, forward output:

$$J = \frac{1}{2N} \sum_{i=1}^{N} \left(z^{(i)} - \hat{z}^{(i)} \right)^2$$

  - Backward output:

$$\frac{\partial J}{\partial z^{(i)}} = \frac{1}{N} \left(z^{(i)} - \hat{z}^{(i)} \right)$$

- L1 loss layer
  - Also commonly used for regression problem, especially when there are many outliers
  - Forward inputs: $z^{(1)}, z^{(2)}, \cdots, z^{(N)}$ and ground-truth $\hat{z}^{(1)}, \hat{z}^{(2)}, \cdots, \hat{z}^{(N)}$, forward output:

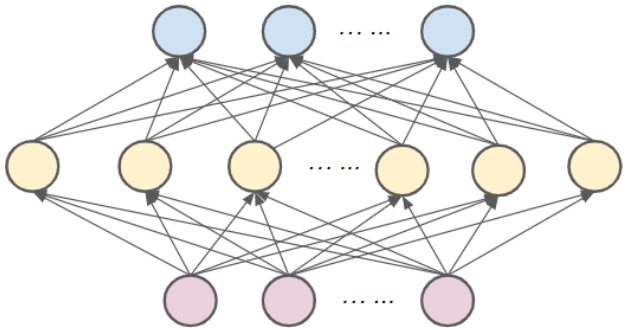
$$J = \frac{1}{N} \sum_{i=1}^{N} \left| z^{(i)} - \hat{z}^{(i)} \right|$$

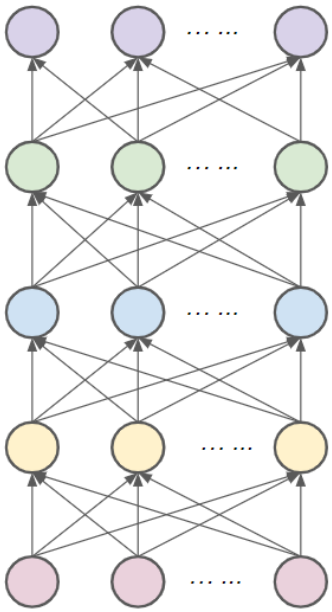
  - Backward output:

$$\frac{\partial J}{\partial z^{(i)}} = \begin{cases} -\frac{1}{N} & \text{if } z^{(i)} - \hat{z}^{(i)} \geq 0 \\ \frac{1}{N} & \text{if } z^{(i)} - \hat{z}^{(i)} < 0 \end{cases}$$

Why do we need "deep" neural networks

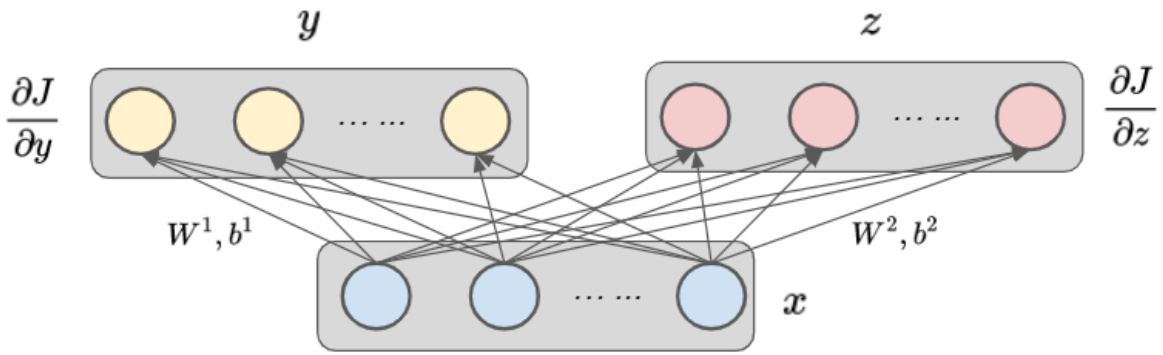
- Theoretically, a three-layer neural network can approximate any non-linear function. Logistic regression/classification can all be considered as a "shallow" three-layer neural network
- Then, why do we need "deep" neural networks?
- If the desired function is very complex, with three-layer neural networks, it might require an exponentially increasing number of neurons in the hidden layers to well approximate the function
- However, with many layers, a small number of neurons in each layer would be enough to approximate the desired function





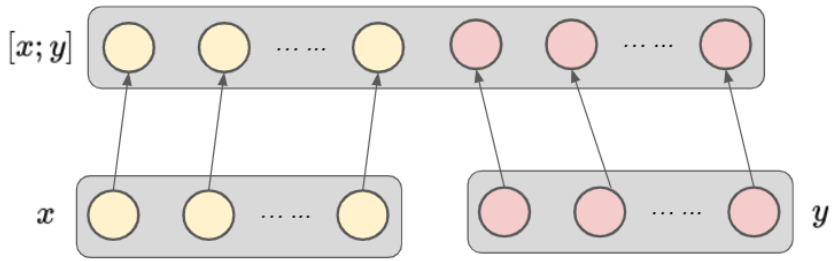
Branching and concatenation

- A group of neurons can be connected by two different fully-connected layers (branches)



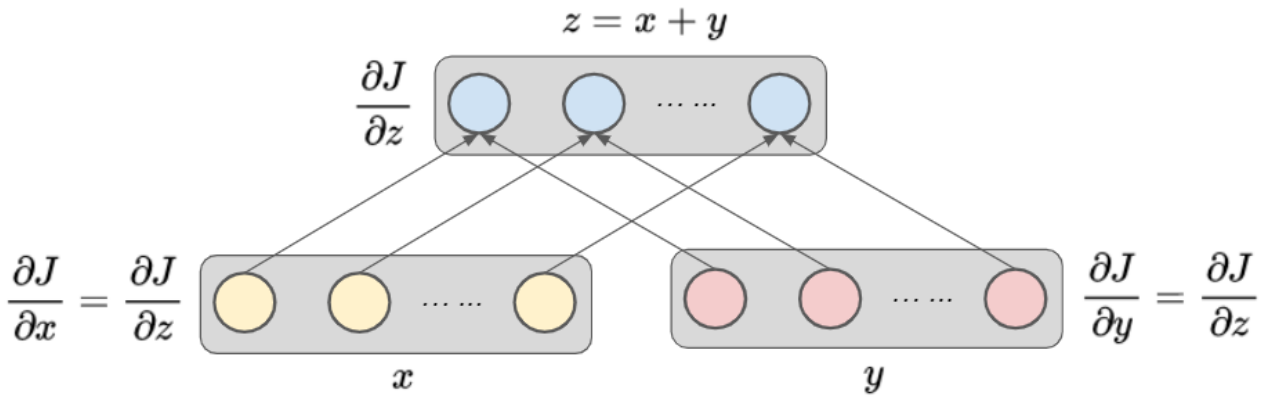
$$\frac{\partial J}{\partial x} = (W^1)^T \frac{\partial J}{\partial y} + (W^2)^T \frac{\partial J}{\partial z}$$

- Two feature vectors (branches) can also concatenate to generate a longer feature vector



Addition of two groups of neurons

- The two vectors of neurons can be added to obtain a group of neurons



Batch Normalization (BN) Layer

- Each dimension of the input feature vectors should be normalized by subtracting the mean over the entire training set and then optionally divided by the standard deviation over the entire training set
- Recall that in mini-batch gradient descent, we train neural networks with mini-batches of samples and each mini-batch might have different feature distributions (named *covariance shift*) because of the small mini-batch size
- To handle different feature distributions in each iteration, the neural networks need to jointly handle feature distribution variations and correctly classify the training samples, which prevent the network from focusing on only learning for classification

Batch Normalization (BN) Layer (cont'd)

- The BN layer normalizes each input feature vector of a mini-batch
- Forward input: feature vector $x \in \mathbb{R}^n$ in a mini-batch $\mathcal{B}$

$$\hat{\mu}_{\mathcal{B}} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} x \quad \text{and} \quad \hat{\sigma}^2_{\mathcal{B}} \leftarrow \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} (x - \mu_{\mathcal{B}})^2 + \epsilon$$

$$\mathrm{BN}(x) = \gamma \odot \frac{x - \hat{\mu}_{\mathcal{B}}}{\hat{\sigma}_{\mathcal{B}} + \epsilon} + \beta \text{ ("}\odot\text{": element-wise multiplication)}$$

- To address the fact that in some cases the activations may actually need to differ from standardized data, BN also introduces learnable scaling coefficients $\gamma \in \mathbb{R}^n$ and offset $\beta \in \mathbb{R}^n$
- We add a small constant $\epsilon > 0$ to the variance estimate to ensure never dividing by zero
- Training:
  - In practice, instead of estimating mean and standard deviation of each mini-batch, we keep a running estimate of the batch feature mean and standard deviation

$$\hat{x}_{\text{new}} = (1 - \text{ momentum }) \times \hat{x}_{\text{old}} + \text{momentum} \times x^{(t)}$$

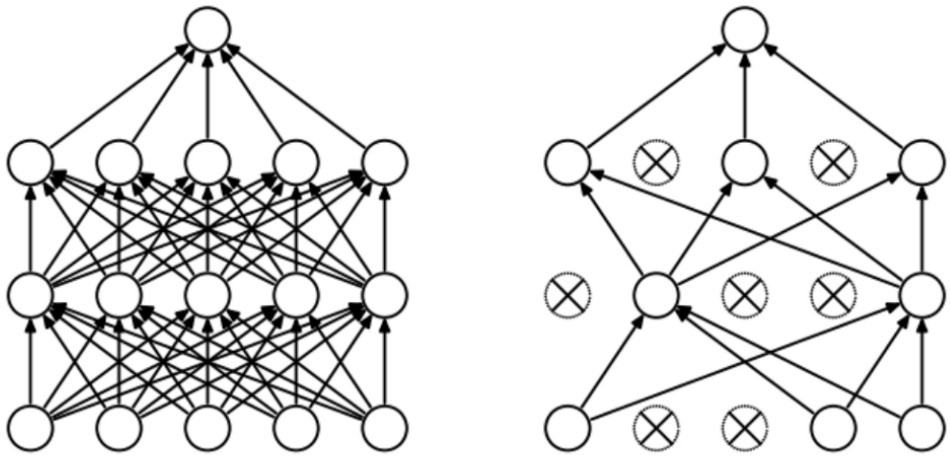
  - $x^{(t)}$ is the statistics of the mini-batch at iteration t. A common choice of momentum is 0.1

Batch Normalization (BN) Layer (cont'd)

- Testing:
  - There are three choices of mean and standard deviation during testing
  - (1) Calculate the mean and standard deviation from the current batch
  - (2) Use the running estimate of mean and standard deviation during training
  - (3) Calculate the mean and standard deviation from the entire training set or a relative large sub-set of the training set layer by layer (rarely used)
- Advantages of using BN layers
  - **Network trains faster**: Each training iteration will actually be slower because of the extra calculations. However, it should converge much more quickly, so training should be faster overall
  - **Allows higher learning rates**: Gradient descent usually requires small learning rates for the network to converge. And as networks get deeper, their gradients get smaller during back propagation so they require even more iterations. Using batch normalization allows us to use much higher learning rates, which further increases the speed at which networks train
  - **Makes weights easier to initialize**: Batch normalization seems to allow us to be much less careful about choosing our initial starting weights
  - **Makes more activation functions viable**: For instance, Sigmoids lose their gradient pretty quickly when used in neural networks

Dropout layer

- Deep neural networks can have many large model capacity because of their deep structures
- They are likely to overfit on small-scale dataset
- Some neurons easily become "inactive" during training, because a small number of other neurons can perform well on the training set
- To mitigate the problem, the dropout layer randomly sets proportion of $p \in [0, 1]$ neurons to zero and force the following the layer to use the remaining neuron responses for completing the prediction task
- General guideline: use after fully-connected layers but **not the topmost** fully-connected layer



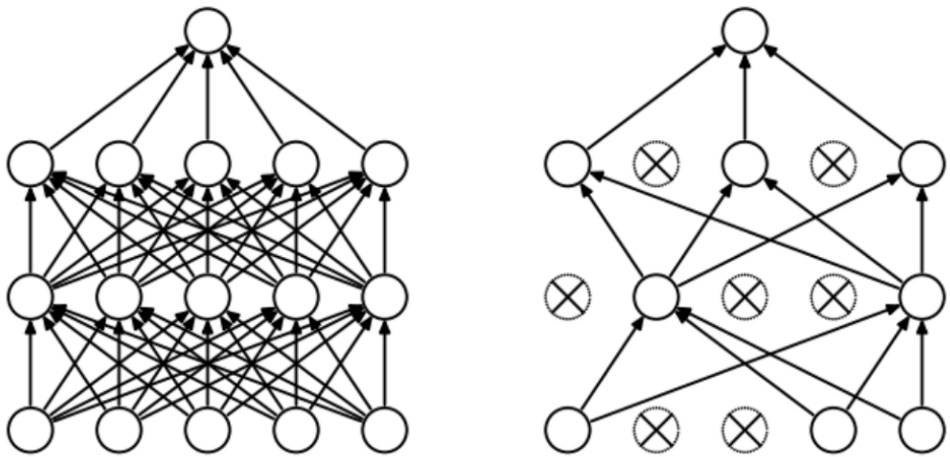
(a) Standard Neural Net (b) After applying dropout.

Dropout layer

- Training:
  - Forward input: dropout ratio p, input feature vector $z = [z_1, z_2, \cdots, z_n]$, forward output: randomly set proportion p of feature values in $[z_1, z_2, \cdots, z_n]$ to zero to obtain y
  - Backward input: $\frac{\partial J}{\partial y}$, backward output:

$$\frac{\partial J}{\partial z_i} = \begin{cases} \frac{\partial J}{\partial z_i} & \text{if } z_i \text{ is not dropped out in forward computation} \\ 0 & \text{if } z_i \text{ is dropped out in forward computation} \end{cases}$$

- Testing/Inference:
  - Forward input: dropout ratio p, input feature vector $z = [z_1, z_2, \cdots, z_n]$; forward output: $[(1-p)z_1, (1-p)z_2, \cdots, (1-p)z_n]$



(a) Standard Neural Net (b) After applying dropout.

Dropout layer

- In general, when using dropout layers, training errors (losses) will **INCREASE**
- For small-scale datasets, dropout layers are effective and **decrease** testing errors
- However, since the dropout layer is designed to prevent overtiffting, it shows **LESS to NONE** effectiveness on large-scale datasets

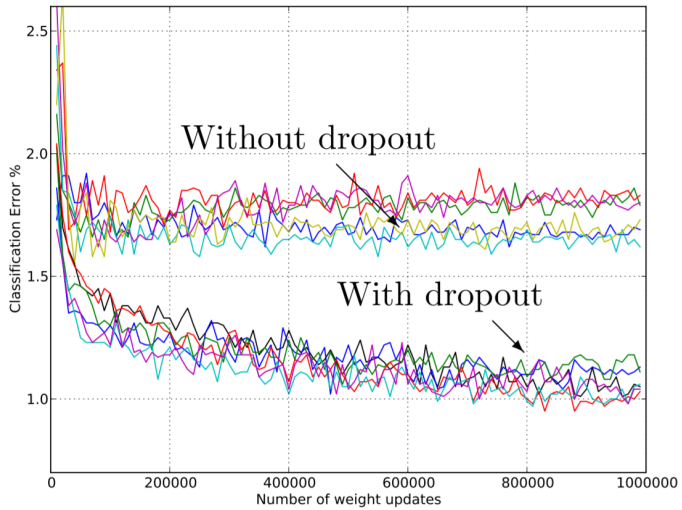
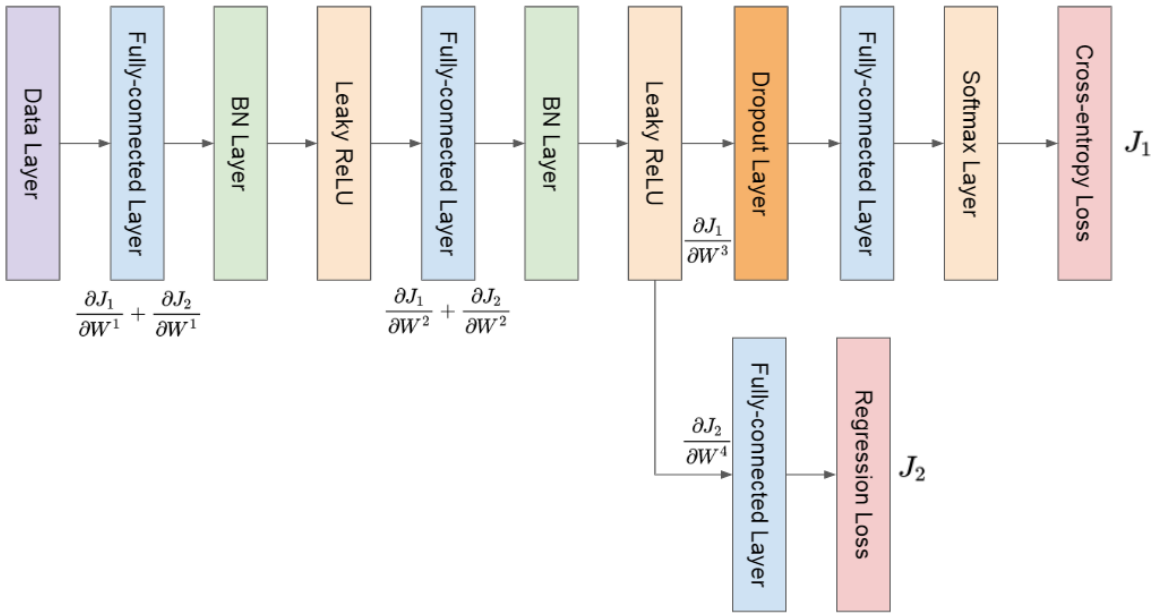


Figure: Test error on MINIST datasets for different architectures with and without dropout. The networks have 2 to 4 hidden layers each with 1024 to 2048 units.

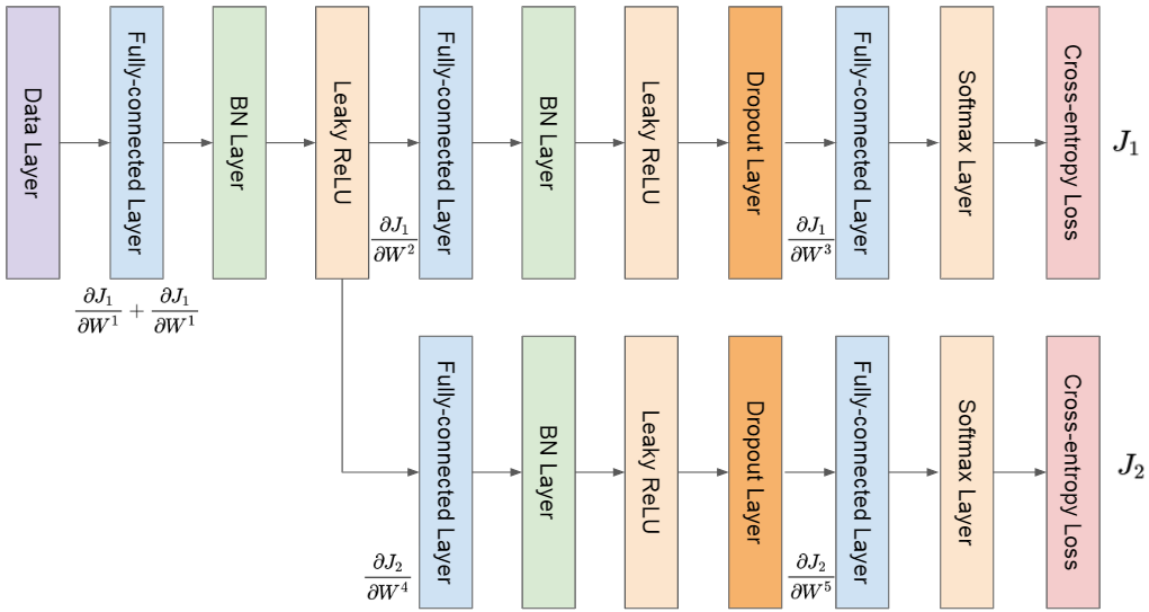
Modern MLPs

- A modern MLP can consist of several fully-connected layers, each of which is followed by a BN layer and then a PReLU or Leaky ReLU non-linearity layer
- Each dimension of the input feature dimension should be normalized by first subtracting the mean and then dividing by the standard deviation
- An MLP can have multiple losses either all at the topmost layer or at different layers



Modern MLPs (cont'd)

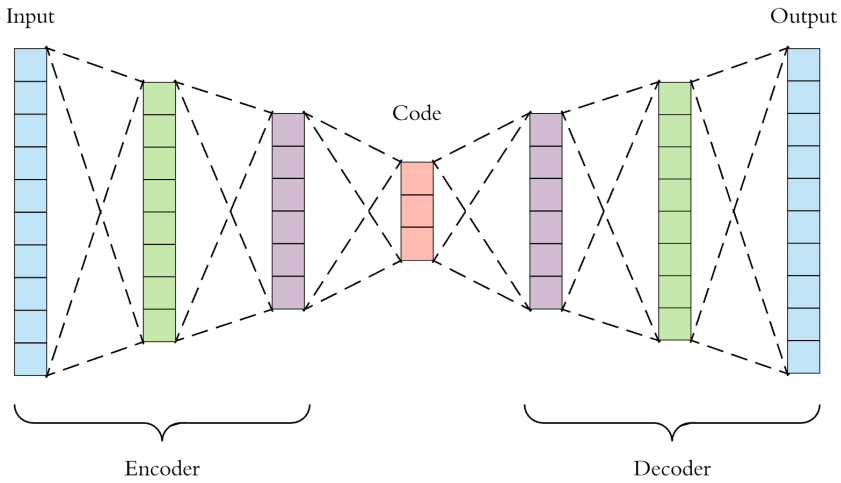
- A modern MLP can consist of several fully-connected layers, each of which is followed by BN layer and then PReLU or Leaky ReLU non-linearity layer
- Each dimension of the *input* feature dimension should be normalized by first subtracting the mean and then dividing by the standard deviation
- An MLP can have multiple losses either all at the topmost layer or at different layers



Autoencoder for unsupervised learning

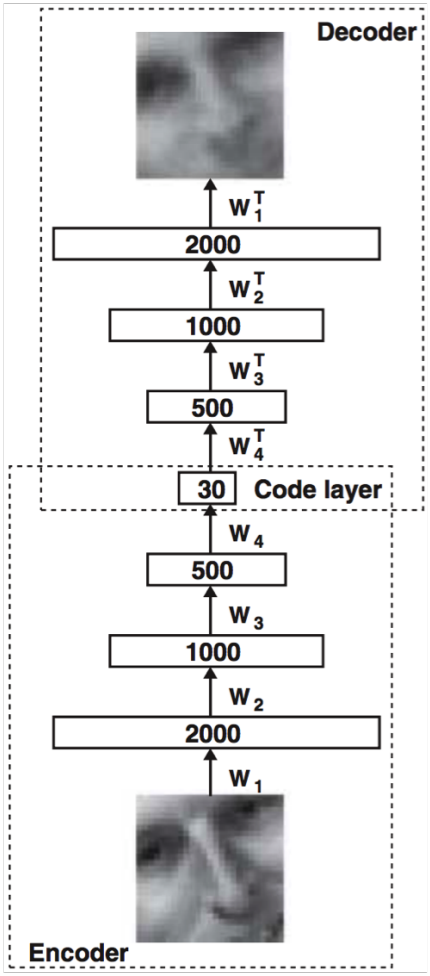
- Autoencoder can be considered as an unsupervised learning method, whose goal is to learn a neural network that is able to encode input high-dimensional feature vectors into low-dimensional feature vectors
- It consists of an encoder and a decoder, which both consists of a series of stacked fully-connected layers
  - Encoder: gradually decreases the number of neurons in each layer
  - Decoder: gradually increases the number of neurons in each layer
  - In most network structure designs, the encoder and decoder have the same number of layers and mirrored number of neurons
- Let $x^{(1)}, x^{(2)}, \cdots, x^{(N)}$ denote the input feature vectors and $\tilde{x}^{(1)}, \tilde{x}^{(2)}, \cdots, \tilde{x}^{(N)}$ denote the output feature vectors. The reconstruction loss function of autoencoder is

$$J = \frac{1}{2N} \sum_{i=1}^{N} \|x^{(i)} - \tilde{x}^{(i)}\|_2^2$$



Autoencoder

- Illustration of the Autoencoder in Hinton's (Science 2006) paper



Autoencoder

- A comparison between PCA (a classifical unsupervised learning method) and autoencoder on learning two-dimensional codes for MINIST digits

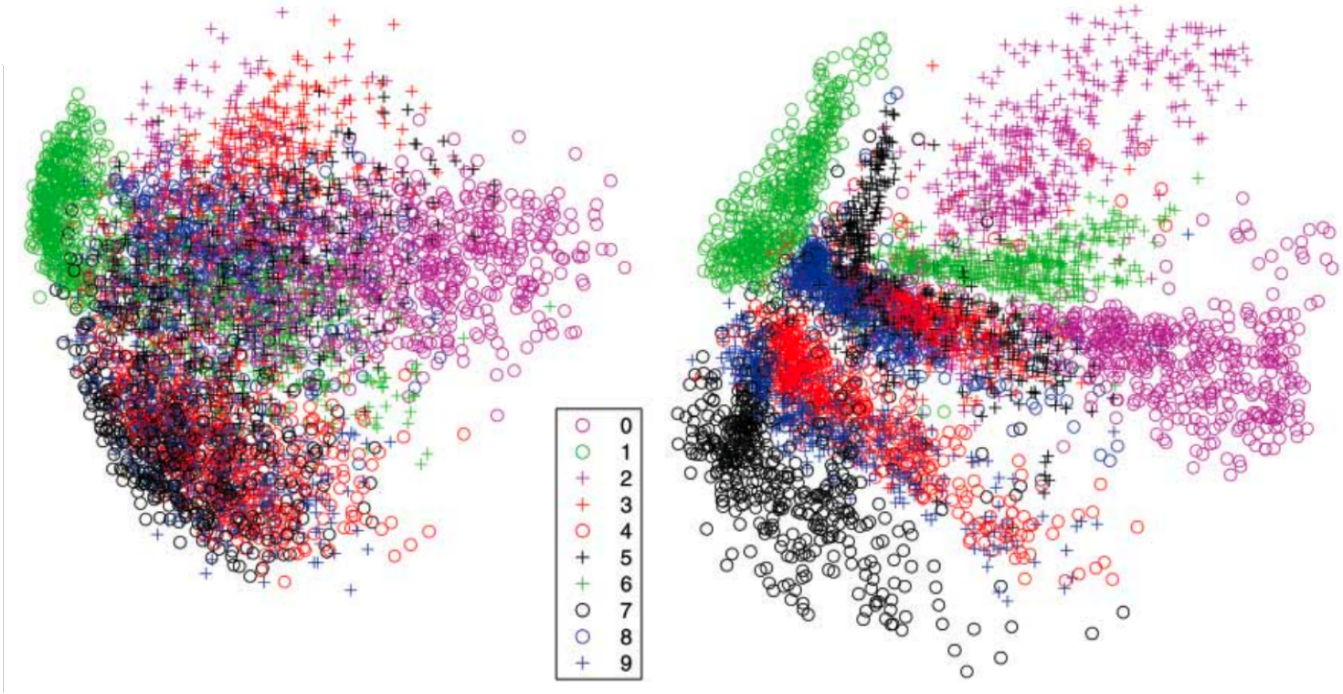


Figure: Left: 2-D codes generated by PCA. Right: 2-D codes generated by autoencoder.

Denoising Autoencoder

- There is also variants of autoencoder. One famous one is denoising autoencoder
- It randomly sets zeros to either inputs or to intermediate feature values to zero and require the autoencoder to reconstruct the clean version of the inputs

